

Predictive Target Pursuit for Autonomous UAVs using RF-DETR with Depth-Aware State Estimation and Physics-Informed Trajectory Prediction

Rylan Malarchick*, Carmen Dimario, Enrique Amaya, Graysen Brinkman,
Sajid Berhane, Jose Castelblanco, Chirag Kumar

Embry-Riddle Aeronautical University, Daytona Beach, FL 32114, United States

*Corresponding author: rylan1012@gmail.com

Abstract

Autonomous UAV target pursuit requires robust detection, state estimation, and control under conditions where visual input degrades or disappears entirely. We present AIRHOUND, an onboard perception-to-control pipeline for single-target drone tracking that runs entirely on an NVIDIA Jetson Orin. The system integrates two detection architectures, RF-DETR (transformer-based) and YOLOv8n (CNN-based), with Intel RealSense D455 stereo depth for 3D state estimation, a 6-state Kalman filter for temporal smoothing, and a physics-informed neural network (PINN) for trajectory prediction during detection dropout. We evaluate both detectors under four synthetic degradation types (Gaussian noise, low light, motion blur, partial occlusion) at five severity levels. RF-DETR retains 75% of its clean-baseline mAP at maximum noise severity, compared to 33% for YOLOv8n, a +39.8 percentage point advantage attributable to the transformer attention mechanism's ability to aggregate global context from uncorrupted image regions. RF-DETR achieves 15 FPS end-to-end on the Jetson Orin after TensorRT FP16 conversion, sufficient for the 10 Hz control loop. The PINN enforces a velocity magnitude constraint ($\|\mathbf{v}\| \leq 10$ m/s) during training, preventing physically implausible extrapolation during multi-second dropouts. The complete pipeline is validated in PX4 hardware-in-the-loop configuration with closed-loop yaw tracking, achieving 58.8 ms mean end-to-end detection latency on the Jetson Orin with TensorRT. All perception and tracking components are modular, configurable at runtime via a single YAML file, and communicate over standard ROS 2 interfaces.

Keywords: UAV target tracking, transformer object detection, physics-informed neural network, depth estimation, Kalman filter, ROS 2, edge computing, degradation robustness

1 Introduction

Autonomous UAV pursuit of airborne targets is constrained by two problems that compound each other: detection accuracy degrades under adverse visual conditions, and reactive control pipelines have no mechanism to maintain tracking when detections disappear. A target drone crossing through shadow, backlight, or turbulence-induced motion blur can cause a CNN-based detector to lose track for hundreds of milliseconds; during that interval, a purely reactive controller has no input and the pursuit vehicle drifts off target. Recovery requires re-acquisition, which may fail if the target has moved outside the camera's field of view.

Prior work on UAV visual tracking has largely addressed these problems independently. Object detection benchmarks evaluate accuracy under clean conditions or with limited augmentation [1, 2]. Kalman filtering provides short-term state extrapolation but assumes constant velocity, producing increasing error during maneuvering targets [3]. Physics-informed neural networks have been applied to fluid dynamics and structural

mechanics [4] but have seen limited adoption in real-time aerial tracking, where inference latency and onboard compute are hard constraints.

This paper presents AIRHOUND, an integrated perception-to-control pipeline for autonomous UAV target pursuit that addresses both problems within a single onboard system. The contributions are:

1. A comparative evaluation of RF-DETR [5] and YOLOv8n [1] under four synthetic degradation types at five severity levels, demonstrating that the transformer architecture retains 75% of clean-baseline mAP at maximum noise severity compared to 33% for the CNN baseline.
2. Depth-fused 3D state estimation using Intel RealSense D455 stereo depth with a 6-state Kalman filter, providing position and velocity estimates in the camera optical frame.
3. A physics-informed neural network for trajectory prediction during detection dropout, trained with a velocity-bounded physics loss that prevents implausible extrapolation.
4. A modular ROS 2 pipeline where all components (detector selection, Kalman filter, PINN, control parameters) are configurable at runtime via a single YAML file, enabling systematic ablation without code changes.
5. End-to-end validation in PX4 hardware-in-the-loop configuration with closed-loop yaw tracking on the NVIDIA Jetson Orin, demonstrating 58.8 ms mean detection latency and continuous tracking through multi-second detection dropouts.

The system runs entirely onboard, with the RF-DETR pipeline achieving 15 FPS end-to-end after TensorRT FP16 conversion on the Jetson Orin (16 GB). The pipeline is designed for the SPIE DS112 track theme of machine learning from challenging data: the degradation evaluation directly characterizes how detection architecture choice affects tracking reliability when visual input quality cannot be guaranteed.

2 Related Work

2.1 Object Detection on Embedded Platforms

Real-time object detection on edge devices has been dominated by the YOLO family of single-stage CNN detectors. YOLOv8 [1] achieves strong accuracy-latency tradeoffs at small model sizes (YOLOv8n: 3.2M parameters), and its TensorRT export path is well-supported on NVIDIA Jetson platforms [6]. However, CNN architectures process features through local receptive fields, making them inherently sensitive to pixel-level corruption from noise, blur, and low-light conditions.

Transformer-based detectors offer an alternative paradigm. DETR [2] introduced end-to-end detection with set prediction, eliminating hand-designed components like non-maximum suppression. RF-DETR [5] builds on this with a DINOv2 [7] vision encoder, achieving competitive accuracy at real-time speeds. The self-attention mechanism aggregates information globally across the input image, which provides a structural advantage when portions of the image are degraded: uncorrupted regions can still contribute to detection. To our knowledge, no prior work has systematically compared transformer and CNN detectors under controlled multi-type degradation specifically for UAV-to-UAV tracking on edge hardware.

2.2 Visual Tracking and State Estimation

Kalman filtering [3] remains the standard approach for temporal state estimation in visual tracking systems. For UAV applications, the constant-velocity model is commonly extended to 3D using stereo or structured-light

depth sensors. The Intel RealSense D455 [8] provides hardware-synchronized RGB-D streams suitable for real-time depth fusion, though stereo depth accuracy degrades beyond 4–6 m and at object boundaries. Mahalanobis gating is a standard technique for rejecting outlier measurements in cluttered environments, preventing spurious detections from corrupting the state estimate.

The limitation of Kalman-based tracking is its reliance on a linear motion model. During extended detection dropout (multiple frames), the constant-velocity extrapolation diverges from the true trajectory if the target is turning, accelerating, or decelerating. This motivates the integration of learned prediction models that can capture nonlinear dynamics.

2.3 Physics-Informed Neural Networks

Physics-informed neural networks (PINNs) incorporate physical laws as soft constraints in the training loss [4]. Originally developed for solving partial differential equations in fluid dynamics and structural mechanics, the PINN framework has been extended to trajectory prediction in autonomous driving and robotics. The key advantage is that physics constraints regularize the network's predictions in regions of state space not well-covered by training data, which is precisely the regime encountered during detection dropout.

For UAV target tracking, the relevant physical constraint is that the target's velocity magnitude is bounded by its flight envelope. Encoding $\|\mathbf{v}\| \leq v_{\max}$ as a penalty term prevents the network from extrapolating trajectories at physically impossible speeds, a failure mode that pure data-driven models exhibit when iteratively feeding predictions back as inputs during multi-second dropouts.

2.4 UAV Visual Servoing and Pursuit

Visual servoing for UAV tracking typically operates in image-based (IBVS) or position-based (PBVS) modes. IBVS controllers compute control signals directly from image features (e.g., pixel error), while PBVS controllers first estimate 3D target position and then compute control in Cartesian space. The AIRHOUND system uses a hybrid approach: yaw control is image-based (pixel error converted to angular error via focal length), while depth-fused 3D estimation enables future extension to position-based altitude and pitch control.

Most UAV tracking systems treat detection and control as independent modules with a thin interface. The contribution of this work is the integration of a predictive component (PINN) between detection and control that anticipates target motion during dropout, enabling the controller to maintain tracking through intervals where a purely reactive system would lose the target.

3 System Architecture

The AIRHOUND system integrates perception, state estimation, and flight control into a modular ROS 2 [9] pipeline deployed on an edge-class companion computer. This section describes the hardware platform, the node-level software architecture, and the unified configuration mechanism that enables systematic ablation of each component.

3.1 Hardware Platform

The airframe is a Holybro X500 quadrotor equipped with three primary avionics components:

- **Flight controller:** Auterion PX4 FMUv6X [10] running the PX4 autopilot with a separate IO coprocessor for PWM output. The FMU provides attitude control, state estimation, and the offboard control interface.

- **Companion computer:** NVIDIA Jetson Orin (16 GB) [6], executing all perception, tracking, and control nodes. The Jetson communicates with the FMU over a dedicated Ethernet link (Jetson 192.168.0.3 $\leftrightarrow$ Pixhawk 192.168.0.4) using the Micro XRCE-DDS bridge [11], which translates between ROS 2 DDS topics and PX4's internal uORB messaging.
- **Camera:** Intel RealSense D455 [8] stereo depth camera, providing synchronized RGB and depth streams at 1280×720 and 30 FPS over USB 3.0.

3.2 Software Architecture

The pipeline is implemented as four ROS 2 Humble nodes connected by typed DDS topics. Each node is a single-responsibility process that can be independently tested, profiled, or replaced:

1. **Perception node** (`detector_node`). Captures RGB frames (either via ROS 2 image topic subscription or direct V4L2 system calls that bypass DDS serialization) and runs TensorRT [12] FP16 inference using the selected detector (RF-DETR [5] or YOLOv8n [1]). When depth is enabled, the node fuses the D455 depth stream with each detection by extracting the median depth within a 5×5 pixel window around the bounding box centre, filtered to the camera's reliable range (0.4–6.0 m). Outputs: `/detections` (`Detection2DArray`), `/perception/target_3d` (`PointStamped`).
2. **Tracking node** (`yaw_error_node`). Subscribes to `/detections` and, when available, the PINN and Kalman state topics. On each frame it selects the best tracking source via the fallback hierarchy described in Section 7.2 and computes a yaw rate command using the pixel-to-angle conversion of Section 7.1. An integrated 6-state constant-velocity Kalman filter [3] with Mahalanobis gating (χ^2 threshold 3.0, reset after 30 consecutive misses) smooths 3D position estimates when depth is available. Outputs: `/yaw_command` (`Float64`), `/tracking/mode` (`Float64`).
3. **PINN prediction node** (`pinn_prediction_node`). A physics-informed neural network [4] trained on SITL trajectory data (Section 6). During detection dropout the node autoregressively predicts the target's future state at 0.1 s horizons, publishing on `/tracking/pinn_predicted` (`PointStamped`) and `/tracking/pinn_active` (`Bool`). After TensorRT conversion the PINN runs in 0.28 ms per inference on the Jetson GPU.
4. **PX4 converter node** (`px4_converter_node`). Bridges the ROS 2 domain and the PX4 autopilot. At 10 Hz it publishes `OffboardControlMode`, `TrajectorySetpoint` (position hold at NED $[0, 0, -5]$ m with yaw override), and `VehicleCommand` messages to the `/fmu/in/*` topics. An automatic startup sequence sends 10 setpoints before commanding offboard mode and arming. A safety watchdog triggers a warning after 5 s without a valid yaw command.

In SITL mode, a *mock detector* replaces the perception node, generating synthetic detections at 30 Hz with configurable target motion (circular, sinusoidal, figure-eight) and optional Gaussian pixel noise. The Gazebo [13] simulator provides the flight dynamics model for the X500 airframe.

3.3 Topic Data Flow

The end-to-end data flow from sensor to actuator proceeds as follows:



All inter-node communication uses ROS 2 DDS with best-effort QoS, prioritizing low latency over guaranteed delivery. History depths range from 1 (PX4 bridge topics) to 5–10 (sensor and inter-node topics) depending on tolerance for message loss. The sole exception is the camera image transport path: at 1280×720 , each uncompressed BGR8 frame is 2.7 MB, which saturates the DDS serialization pipeline in Python and limits throughput to approximately 1.5 FPS. The V4L2 direct capture path (Section 8) bypasses this bottleneck entirely, restoring the full TensorRT-limited throughput of 15 FPS.

3.4 Unified Configuration

All tunable parameters (detector type and model path, camera resolution, Kalman filter noise covariances, PINN model and prediction horizon, yaw rate limits, PX4 hover setpoint, and SITL mock-detector motion profiles) are consolidated in a single YAML configuration file (`airhound.yaml`). A mode selector (`sim` or `flight`) determines which nodes and parameter subsets are active, and a top-level launch script (`launch_airhound.sh`) orchestrates the Micro XRCE-DDS agent, PX4 SITL (if applicable), and the ROS 2 launch graph in a single command. This design enables systematic ablation of any component (swapping detectors, disabling the PINN, or adjusting control gains) without modifying source code.

4 RF-DETR Detection and Degradation Robustness

4.1 Model Selection

We evaluate two object detection architectures for onboard UAV detection: YOLOv8n [1], a single-stage CNN-based detector, and RF-DETR [5], a transformer-based detection model built on a DINOv2 [7] vision encoder with a DETR-style [2] decoder. The selection of these architectures is motivated by the distinct computational paradigms they represent: YOLOv8n processes features through convolutional layers with local receptive fields, while RF-DETR uses self-attention mechanisms that aggregate global context across the input image.

This architectural distinction has implications for robustness under degraded visual conditions. Convolutional networks rely on local spatial patterns that are directly disrupted by pixel-level noise, low-light conditions, and motion blur. In contrast, the attention mechanism in transformer-based detectors can integrate information from unaffected regions of the image, potentially maintaining detection accuracy when portions of the input are corrupted.

4.2 Training Configuration

Both models are trained on a drone detection dataset comprising 13,869 training images and 1,983 validation images sourced from the Roboflow Drones dataset [14], annotated with a single class (`drone`).

The RF-DETR base variant is used, consisting of a DINOv2 windowed-small encoder with a 3-layer decoder (256 hidden dimension, 300 queries). The model is initialized from COCO-pretrained weights and fine-tuned for 50 epochs with the Adam optimizer (learning rate 1×10^{-4} , weight decay 1×10^{-4}), gradient clipping at 0.1 max norm, and 5 warmup epochs. Mixed-precision training (FP16) is used with an effective batch size of 4 on an NVIDIA RTX 4070 (12 GB VRAM). Data augmentation includes mosaic and mixup.

YOLOv8n is trained using the Ultralytics framework with default hyperparameters on the same dataset for a fair comparison.

4.3 Edge Deployment via TensorRT

For real-time inference on the NVIDIA Jetson Orin (16 GB) [6], both models are converted to TensorRT [12] FP16 engines. The RF-DETR conversion pipeline proceeds as:

1. PyTorch checkpoint $\rightarrow$ ONNX (opset 17, input 560×560)
2. ONNX graph simplification via `onnx-simplifier`
3. ONNX $\rightarrow$ TensorRT engine via `trtexec` with FP16 and 4 GB workspace

The resulting RF-DETR TensorRT engine is 57 MB. End-to-end pipeline throughput (including V4L2 capture and pre/post-processing) is approximately 15 FPS at 1280×720 on the Jetson Orin, compared to approximately 1 FPS for the ONNX model on CPU. YOLOv8n TensorRT achieves higher throughput at its native 640×640 input resolution. Both engines are sufficient for closed-loop control at the 10 Hz offboard setpoint rate required by PX4.

4.4 Synthetic Degradation Pipeline

To evaluate robustness under challenging visual conditions, the central theme of the SPIE DS112 track, we apply synthetic degradation to the validation set. Four degradation types are evaluated at five severity levels ($s \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$), where $s = 0.0$ represents the clean baseline:

- **Gaussian noise:** Additive noise with standard deviation $\sigma = s \cdot 50$ (pixel intensity scale 0–255), simulating sensor noise in low-SNR conditions.
- **Low light:** Multiplicative brightness reduction by factor $(1 - 0.9s)$, simulating twilight or shadowed environments.
- **Motion blur:** Directional kernel blur with kernel size $k = \lfloor 20s \rfloor + 1$ pixels, simulating camera or target motion artifacts.
- **Partial occlusion:** Random rectangular patches covering up to $s \cdot 50\%$ of the bounding box area, simulating environmental obstructions.

Each degradation is applied independently to all 1,983 validation images, yielding $2 \times 4 \times 5 = 40$ unique evaluation conditions (2 models $\times$ 4 degradation types $\times$ 5 severities).

4.5 Degradation Robustness Results

4.5.1 Clean Baseline

On clean (undegraded) images, both models achieve comparable performance (Table 1). YOLOv8n holds a marginal advantage in mAP@0.5 (0.957 vs. 0.953) and recall (0.971 vs. 0.931), while RF-DETR achieves higher precision (0.949 vs. 0.922). This establishes that any performance differences under degradation are attributable to architectural robustness rather than baseline capability gaps.

Table 1: Clean baseline detection performance on 1,983 drone images.

Metric	YOLOv8n	RF-DETR	Δ
mAP@0.5	0.957	0.953	-0.005
mAP@0.5:0.95	0.584	0.541	-0.043
Precision	0.922	0.949	+0.027
Recall	0.971	0.931	-0.040
F1 Score	0.946	0.940	-0.006

4.5.2 Performance Under Degradation

Figure 1 shows mAP@0.5 as a function of severity level for each degradation type. Two patterns emerge clearly:

1. **Gaussian noise and low light:** RF-DETR maintains significantly higher accuracy as severity increases. At maximum Gaussian noise severity ($s = 1.0$), RF-DETR achieves 0.716 mAP@0.5 compared to YOLOv8n's 0.318, a +39.8 percentage point advantage. Under maximum low-light conditions, the advantage is +39.4 points (0.670 vs. 0.275). RF-DETR retains 75% and 70% of its clean-baseline performance at maximum noise and low-light severity respectively, while YOLOv8n retains only 33% and 29% (Figure 2).
2. **Motion blur and occlusion:** Both models are significantly impacted by severe motion blur, with mAP dropping below 0.20 at $s \geq 0.75$. RF-DETR maintains a moderate advantage at low blur severity (+13.9 points at $s = 0.25$), but neither architecture alone can sustain detection at high blur levels. Occlusion performance is comparable, with both models handling partial obstructions gracefully.

The advantage heatmap (Figure 3) provides a complete summary across all conditions. The delta mAP values confirm that RF-DETR's advantage scales with severity: near-zero difference on clean data, growing to $\sim +0.40$ under severe noise and low-light conditions.

Table 2 summarizes the average mAP@0.5 across all severity levels for each degradation type. RF-DETR achieves +17.8 and +13.4 average mAP advantage under Gaussian noise and low light respectively.

Table 2: Average mAP@0.5 across all severity levels (0.0–1.0) by degradation type.

Degradation	YOLOv8n	RF-DETR	Δ
Gaussian Noise	0.682	0.860	+0.178
Low Light	0.716	0.850	+0.134
Motion Blur	0.404	0.455	+0.051
Occlusion	0.889	0.890	+0.001

4.5.3 Implications for System Design

These results inform two key architectural decisions:

1. **Detector selection:** RF-DETR is selected as the primary detector for flight operations. Its robustness advantage under noise and lighting variation, conditions frequently encountered in outdoor UAV operations, outweighs YOLOv8n's marginal clean-data and speed advantages. Both models are retained in the pipeline, switchable via configuration, to enable comparative flight testing.

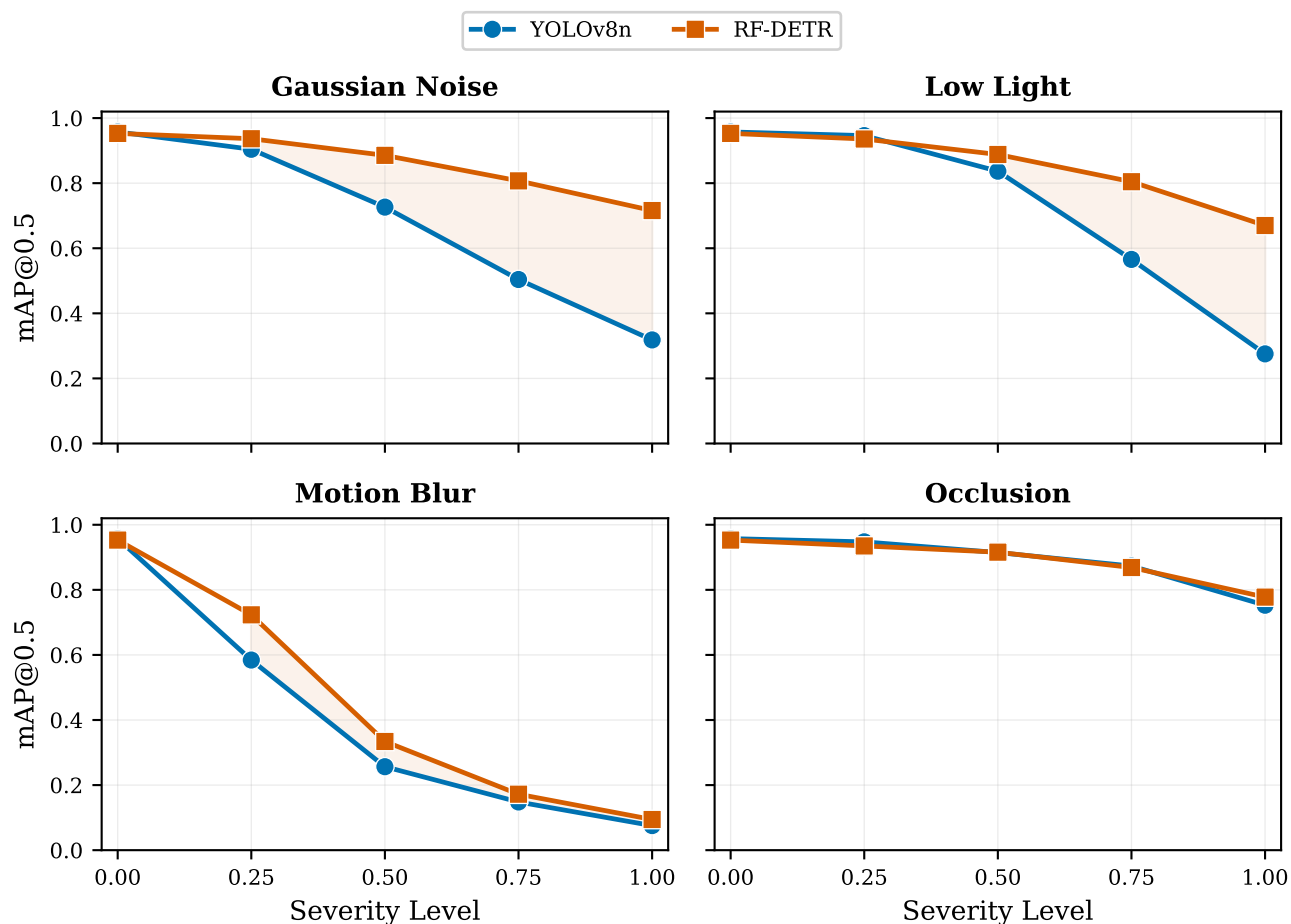


Figure 1: Detection accuracy (mAP@0.5) under four degradation types at increasing severity. RF-DETR (orange squares) demonstrates substantially higher robustness than YOLOv8n (blue circles) under Gaussian noise and low-light conditions. Shaded region indicates RF-DETR advantage.

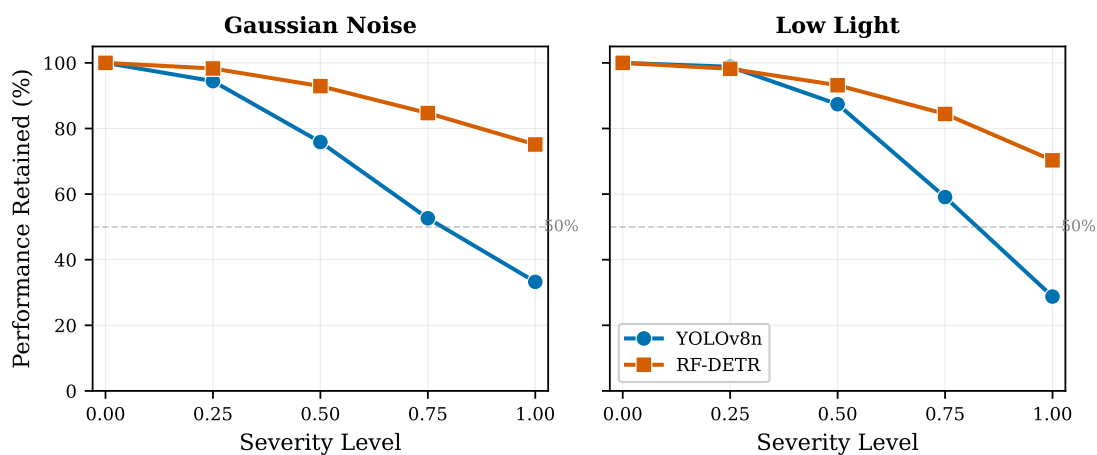


Figure 2: Percentage of clean-baseline mAP@0.5 retained under increasing degradation severity for Gaussian noise and low light, the two conditions with the largest architectural performance gap.

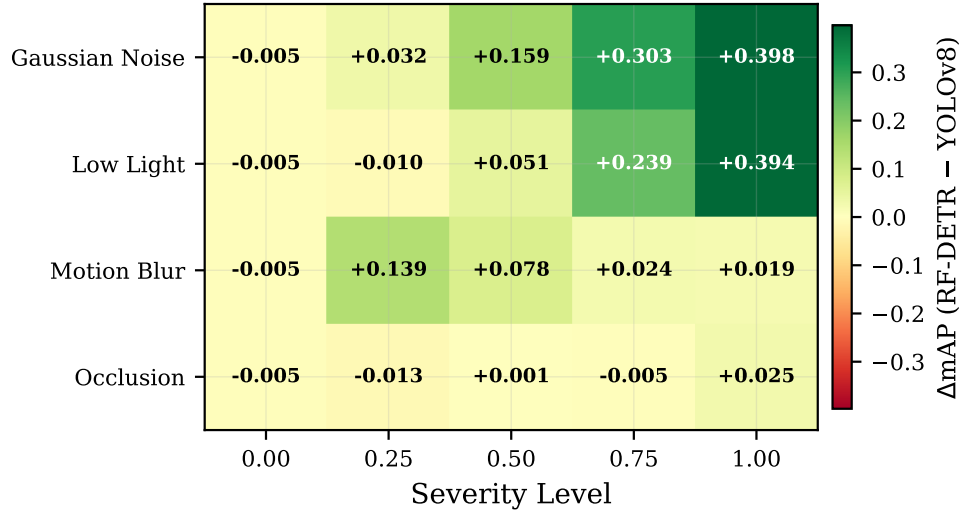


Figure 3: Heatmap of mAP@0.5 difference (RF-DETR – YOLOv8n) across all degradation types and severity levels. Green cells indicate RF-DETR outperformance.

2. **Complementary prediction:** The severe performance degradation of both models under motion blur (<0.20 mAP at $s \geq 0.75$) motivates the integration of a predictive tracking component. Detection dropouts during blur events must be bridged by Kalman filtering and PINN-based trajectory prediction (Sections 5 and 6).

5 Depth Integration and 3D State Estimation

The RF-DETR detector provides target localization in the image plane, but the downstream tracking and control layers require a metric estimate of target position. AIRHOUND therefore fuses RGB detections with depth measurements from the Intel RealSense D455 [8], allowing valid detections to be projected into 3D in the camera optical frame. These observations are then filtered by a constant-velocity 6-state Kalman filter, which produces a smoothed estimate of target position and velocity for short-horizon extrapolation and downstream PINN prediction.

5.1 Depth Extraction from the D455

For each detection, the perception node samples the aligned depth image at the centre of the predicted bounding box. Because an individual stereo-depth pixel can be unstable near object boundaries or in weak-texture regions, AIRHOUND applies a small windowed median filter around the bounding-box centre rather than relying on a single raw sample. Only nonzero depth values are retained, and the median of the valid samples is used as the representative range for that detection.

The D455 depth image is received as a 16-bit stream in millimetres and converted to metres using a scale factor of 0.001. To suppress invalid near-field and long-range measurements, the resulting depth is filtered by a valid operating interval of 0.4–6.0 m, consistent with the reliable range assumed elsewhere in the tracking stack. If no valid in-range value remains after this filtering step, the detection is treated as having no usable depth for 3D state estimation on that frame.

5.2 3D Projection via the Pinhole Model

Given a valid depth estimate, the 2D detection is lifted into 3D using the standard pinhole camera model. Let (u, v) denote the bounding-box centre in pixels, let (f_x, f_y, c_x, c_y) denote the camera intrinsics from the `/camera/camera_info` topic, and let z denote the filtered depth in metres. The corresponding 3D target position is computed as

$$x = \frac{(u - c_x)z}{f_x}, \quad y = \frac{(v - c_y)z}{f_y}, \quad z = z \quad (1)$$

The resulting coordinates are expressed in the ROS optical frame convention, with X to the right, Y downward, and Z forward. The best valid 3D observation is published on `/perception/target_3d` as a `PointStamped`, providing a metric target measurement for the tracking layer.

5.3 Six-State Kalman Filter

The projected 3D observations are smoothed by a constant-velocity Kalman filter [3] with state

$$\mathbf{x}_k = [x, y, z, v_x, v_y, v_z]^\top \quad (2)$$

where (x, y, z) denotes target position and (v_x, v_y, v_z) denotes target velocity in the camera optical frame. The filter is initialized from the first valid 3D measurement, with position set directly from the observation and velocity initialized to zero.

Assuming constant velocity over a short sampling interval Δt , the transition model is

$$\mathbf{x}_{k+1} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_k + \mathbf{w}_k \quad (3)$$

where $\mathbf{w}_k$ is the process noise. In the current implementation, the process covariance is diagonal with position entries scaling as Δt^2 and velocity entries scaling as Δt , reflecting higher uncertainty growth in position during longer prediction intervals. Since the perception stack directly observes only position, the measurement model is

$$\mathbf{z}_k = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \mathbf{x}_k + \mathbf{v}_k \quad (4)$$

where $\mathbf{z}_k = [x, y, z]^\top$ is the incoming 3D observation and $\mathbf{v}_k$ is the measurement noise.

For each update, the filter predicts the state forward using the elapsed interval since the previous measurement and then applies a correction using the new 3D observation if that observation passes the gating test described below. The resulting 6-state estimate is published on `/tracking/kalman_state` as a `Float64MultiArray`, which is then consumed by the PINN prediction node described in Section 6.

5.4 Mahalanobis Gating and Filter Reset

To reduce the impact of spurious depth measurements, AIRHOUND applies Mahalanobis gating within the 3D Kalman filter before the correction step. Let the innovation residual be

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\mathbf{x}_k \quad (5)$$

and let the innovation covariance be

$$\mathbf{S}_k = \mathbf{H}\mathbf{P}_k\mathbf{H}^\top + \mathbf{R} \quad (6)$$

where $\mathbf{P}_k$ is the state covariance and $\mathbf{R}$ is the measurement-noise covariance. The implementation evaluates the Mahalanobis distance

$$d_k = \sqrt{\mathbf{y}_k^\top \mathbf{S}_k^{-1} \mathbf{y}_k} \quad (7)$$

and accepts the measurement only if d_k is below a configurable threshold.

In the current implementation, the default gating threshold is 3.0. Measurements with $d_k < 3.0$ are accepted by the Kalman update, while larger residuals are rejected as outliers. Rejected measurements increment a missed-update counter maintained by the gated filter, and the filter resets after more than 30 consecutive misses. At a nominal 30 Hz update rate, this corresponds to approximately 1 s without an accepted 3D correction.

This gating step is particularly important for stereo depth, where the bounding-box centre may occasionally fall on unstable disparity regions or on background pixels instead of the target body. During short interruptions, the most recent valid Kalman state can still support constant-velocity extrapolation in the downstream tracking logic. However, the reset limit prevents stale state from being propagated indefinitely when valid 3D corrections are not recovered.

5.5 Depth Quality Monitoring

In addition to publishing the 3D target estimate, the perception node reports lightweight diagnostics describing the health of the depth stream. A binary availability signal is published on `/perception/depth_available`, while `/perception/depth_quality` reports the ratio of detections in the current frame that produced valid in-range depth values. These topics allow downstream modules and offline analysis tools to distinguish detector dropouts from degraded depth sensing.

This separation is useful because failures in stereo depth and failures in 2D detection are not equivalent. A detector may still localize the target in the image while the depth estimate becomes unreliable due to low texture, boundary mismatch, or range limitations. By exposing depth quality explicitly at the ROS 2 interface level, AIRHOUND preserves a clean separation between perception, tracking, and control while still allowing later stages to respond appropriately when metric depth becomes unavailable.

6 Physics-Informed Trajectory Prediction

When a detection dropout begins, the Kalman filter's constant-velocity extrapolation diverges progressively from the true trajectory; a target executing a banked turn or deceleration will be several meters from the Kalman prediction within one to two seconds. To bridge these intervals, AIRHOUND integrates a physics-informed neural network that replaces the Kalman extrapolation as the source of tracking setpoints during dropout. The PINN is designed around two constraints imposed by the deployment context: inference must complete within the 33 ms budget of the 30 Hz control loop on the Jetson Orin, and predicted trajectories must remain within the target drone's physical flight envelope regardless of how many autoregressive steps are chained together.

6.1 Network Architecture

The PINN is a fully-connected multi-layer perceptron (MLP) that maps the current 6-state estimate together with a prediction horizon to a future state:

$$f_\theta : [\mathbf{x}, \mathbf{v}, \Delta t] \in \mathbb{R}^7 \mapsto [\mathbf{x}', \mathbf{v}'] \in \mathbb{R}^6 \quad (8)$$

where $\mathbf{x} = [x, y, z]^\top$ is the target's 3D position expressed in the camera optical frame (X right, Y down, Z forward/depth), $\mathbf{v} = [v_x, v_y, v_z]^\top$ is the velocity vector from the Kalman filter, and Δt is the requested prediction horizon in seconds. The output $[\mathbf{x}', \mathbf{v}']$ is the predicted position and velocity at time $t + \Delta t$.

The architecture consists of three hidden layers of 128 units each with ReLU activations, giving a 7–128–128–128–6 topology with approximately 35 k trainable parameters. Weights are initialized with Xavier uniform initialization for training stability [15]. An MLP is deliberately preferred over recurrent architectures: the absence of hidden state eliminates the need for state-reset logic on detection recovery, and the small parameter count keeps TensorRT-compiled inference below 1 ms on the Jetson Orin. All inputs and outputs are standardized channel-wise to zero mean and unit variance using statistics computed from the full training corpus and saved alongside the model weights for consistent normalization at inference time. The full architecture is illustrated in Figure 4.

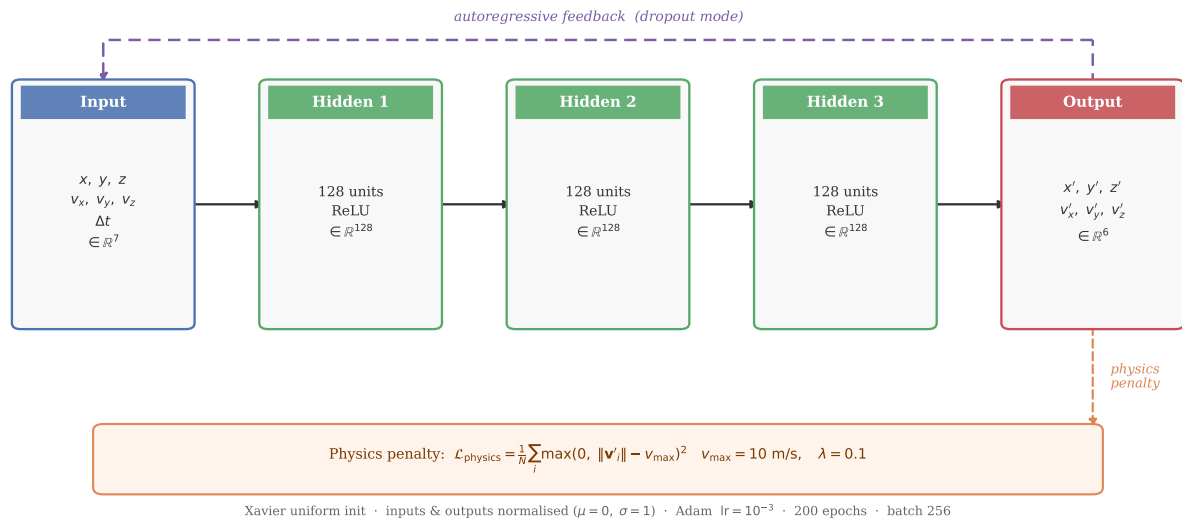


Figure 4: PINN architecture. The 7-dimensional input (position, velocity, and prediction horizon Δt) passes through three hidden layers of 128 ReLU units before producing a 6-dimensional predicted state. During training a physics penalty enforces $\|\mathbf{v}'\| \leq 10$ m/s on the output velocities. In deployment (dropout mode) the predicted state is fed back autoregressively as the next input at 30 Hz.

6.2 Physics-Informed Training Loss

The model is trained by minimizing a composite loss that combines a supervised data term with a physics constraint penalty:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{MSE}} + \lambda \underbrace{\mathcal{L}_{\text{physics}}}_{\text{velocity bound}} \quad (9)$$

$\mathcal{L}_{\text{data}}$ is the mean squared error between the predicted and ground-truth next states, computed in normalized space. $\mathcal{L}_{\text{physics}}$ enforces the target drone's operational flight-envelope constraint as a one-sided quadratic hinge penalty on the predicted speed:

$$\mathcal{L}_{\text{physics}} = \frac{1}{N} \sum_{i=1}^N \max(0, \|\mathbf{v}'_i\| - v_{\text{max}})^2 \quad (10)$$

The physics loss is evaluated in physical (denormalized) units so that $v_{\max} = 10$ m/s applies directly to predicted velocities in metres per second. The hinge formulation is intentional: the penalty is zero everywhere within the feasible velocity regime, so the physics term does not compete with the data term during normal training; it activates only when predictions reach physically implausible speeds. The weighting $\lambda = 0.1$ is chosen to allow tight data fitting while strongly penalizing constraint violations.

The physics constraint becomes critical during autoregressive deployment. At inference the PINN is applied iteratively; the predicted state at step k is fed back as the input state at step $k+1$, so small velocity errors compound over successive steps. A pure data-driven network trained without the velocity bound can produce predictions that accelerate without limit across a multi-second dropout; the hinge penalty prevents this by ensuring that no individual prediction step can push the velocity above the envelope bound, bounding the error growth of the autoregressive chain.

6.3 Training Configuration and Data

Training trajectories are extracted from PX4 SITL simulations (Gazebo, X500 quadrotor model) recorded during closed-loop pursuit runs. The dataset is constructed by building all state-transition pairs $(s_i, \Delta t_{ij}, s_j)$ from each trajectory where $s = [x, y, z, v_x, v_y, v_z]^T$ and the time separation $\Delta t_{ij} \in [0.03, 1.0]$ s. Sampling over a range of horizons rather than a fixed step teaches the network to predict accurately at the 0.1 s deployment horizon while remaining well-conditioned at longer horizons that may arise during extended dropout. Velocities are computed by central finite differences on 30 Hz position sequences, with a moving-average window applied to suppress measurement noise. Samples with depth outside the D455 reliable range (0.4–6.0 m) or with computed speed exceeding $v_{\max}$ are discarded as likely sensor artifacts. The dataset covers straight-flight segments, banked turns, and approach/depart maneuvers to ensure generalization across the target’s operational maneuvering envelope.

The model is trained for 200 epochs with the Adam optimizer ($\text{lr} = 1 \times 10^{-3}$, weight decay 1×10^{-5}), batch size 256, gradient clipping at max norm 1.0, and a ReduceLROnPlateau scheduler (factor 0.5, patience 20 epochs). Fifteen percent of the dataset is held out as a validation split. On the synthetic SITL dataset the trained model achieves a mean position prediction error of 22 cm with zero velocity-constraint violations on the validation set, confirming that the physics penalty successfully eliminates implausible predictions without degrading trajectory accuracy.

6.4 ROS 2 Deployment and Dropout Recovery

The trained model is deployed as `pinn_prediction_node` within the ROS 2 tracking stack. Three inference backends are supported at runtime via the `model_path` parameter: a native PyTorch checkpoint (`.pth`) for development, an ONNX Runtime session (`.onnx`) for cross-platform portability, and a TensorRT FP16 engine (`.engine`) for Jetson deployment. On the Jetson Orin the TensorRT backend sustains 3,045 inferences/s (0.33 ms per call including memory transfer), a $7\times$ speedup over the ONNX Runtime path, consuming less than 1% of the 33 ms control-loop budget and leaving the remaining budget entirely for the detector and Kalman update.

The node’s dropout-recovery logic operates as follows. It subscribes to `/tracking/kalman_state` (`Float64MultiArray, [x, y, z, v_x, v_y, v_z]`), which carries the Kalman filter’s latest smoothed state, and monitors the arrival time of each `/perception/target_3d` detection message. When the interval since the last valid detection exceeds the `dropout_timeout` threshold (default 0.1 s, approximately three missed frames at 30 Hz), PINN prediction mode activates:

1. The most recent Kalman state seeds the first PINN inference with $\Delta t = 0.1$ s.

2. The predicted state $[\mathbf{x}', \mathbf{v}']$ is published on `/tracking/pinn_predicted` (PointStamped) for consumption by the yaw-control node, and simultaneously stored as the input state for the next timer tick.
3. This autoregressive loop repeats at 30 Hz, advancing the predicted trajectory one step per tick without any external measurement input.
4. A Boolean flag on `/tracking/pinn_active` allows downstream nodes to distinguish PINN-predicted setpoints from direct-detection setpoints.
5. On detection recovery the PINN deactivates immediately, the autoregressive chain resets, and the Kalman filter re-initializes from the next measured observation.
6. A configurable `max_prediction_time` caps the total unsupported prediction duration, providing a safety bound against indefinite extrapolation in loss-of-sight scenarios.

In hardware-in-the-loop testing (Sim C, Table 3), the PINN maintains continuous yaw tracking commands through 16 intentional dropout events spanning up to 16.3 s each. Over the 180-second run, 77.5% of total tracking time is spent in PINN prediction mode and only 0.4% in the zero-command state. By contrast, the configurations without the PINN (Sim A and Sim B) spend 68–72% of equivalent tracking time in the zero-command state, where the controller receives no targeting input. The PINN therefore functions as the primary mechanism through which the system sustains pursuit commands across multi-second detection gaps, and its bounded velocity predictions ensure that the yaw commands it generates remain physically consistent with the target’s expected motion throughout the dropout interval.

7 Anticipatory Control

The control layer translates the perception and tracking outputs into PX4-compatible flight commands. Two ROS 2 nodes implement the full pipeline: a *yaw_error_node* that selects the best available tracking source on every frame and computes a yaw rate command, and a *PX4_converter_node* that wraps that command into the offboard control messages expected by the PX4 autopilot [10]. All tunable parameters are exposed through the unified `airhound.yaml` configuration file, enabling systematic ablation of each control component without code changes.

7.1 Yaw Error Computation

The yaw error node (*yaw_error_node*) subscribes to the detector’s `/detections` topic and converts the highest-confidence bounding box centre into an angular yaw rate command. Given a detection with horizontal pixel coordinate u and camera intrinsic focal length f_x , the angular error is

$$e_\psi = \frac{u - c_x}{f_x} \quad (11)$$

where c_x is the principal point. Camera intrinsics are obtained at runtime from the `/camera/camera_info` topic published by the RealSense D455 driver; if the topic is unavailable (e.g. in simulation), the node falls back to default values for the D455 at 1280×720 ($f_x = 615$, $c_x = 640$).

The angular error is passed through a proportional controller with a deadband of ± 0.01 rad and a symmetric rate limit of ± 1.0 rad/s:

$$\dot{\psi}_{\text{cmd}} = \begin{cases} 0 & \text{if } |e_\psi| < 0.01 \\ \text{clip}(e_\psi, -1.0, +1.0) & \text{otherwise} \end{cases} \quad (12)$$

The deadband suppresses oscillatory commands when the target is near-centred, while the rate clamp maps to approximately $\pm 57.3^\circ/\text{s}$, within the safe yaw rate envelope of the X500 airframe.

When a 3D position estimate is available from the Kalman filter (Section 5) or the PINN (Section 6) instead of a raw pixel detection, the yaw command is computed as $e_\psi = \text{atan2}(x, z)$ in the camera optical frame (X right, Z forward), providing a geometrically correct bearing estimate that accounts for target depth.

7.2 Dropout Recovery: Fallback Hierarchy

When no detection arrives on a given frame, the yaw error node evaluates a prioritized fallback hierarchy to select the best available tracking source. The hierarchy is traversed top-to-bottom until a valid command is produced:

1. **PINN prediction** (mode 3). If the PINN node is enabled (`enable_pinn: true`) and the dropout duration exceeds the activation threshold (`dropout_timeout`, default 0.1 s), the most recent PINN-predicted 3D position from `/tracking/pinn_predicted` is converted to a yaw command via the $\text{atan2}(x, z)$ bearing computation. The PINN provides bounded, physics-informed extrapolation and is the preferred fallback for multi-second dropouts (Section 6).
2. **Kalman extrapolation** (mode 2). If the Kalman filter is enabled and initialized, the constant-velocity prediction $\hat{\mathbf{x}}(t + \Delta t) = \mathbf{x} + \mathbf{v} \Delta t$ provides a short-term extrapolation of the target position. A Mahalanobis gating counter tracks consecutive missed updates; after 30 misses (approximately 1 s at 30 Hz) the filter resets, preventing stale state from producing divergent commands.
3. **Hold-then-decay** (modes 4/5). When neither PINN nor Kalman is available, the node holds the last commanded yaw rate for a configurable `hold_time` (default 0.3 s), then linearly decays the command to zero over `decay_time` (default 1.0 s):

$$\dot{\psi}(t) = \begin{cases} \dot{\psi}_{\text{last}} & t_d \leq t_h \\ \frac{t_h + t_c - t_d}{t_c} \dot{\psi}_{\text{last}} & t_h < t_d \leq t_h + t_c \\ 0 & t_d > t_h + t_c \end{cases} \quad (13)$$

where t_d is the time since last detection, t_h is the hold time, and t_c is the decay time. This ensures that a momentary dropout does not immediately zero the command (preserving tracking momentum), while guaranteeing that stale commands are bounded and converge to zero within 1.3 s.

4. **Zero command** (mode 0). If the decay interval has elapsed or no detection has ever been received, the node publishes a zero yaw rate.

A numeric tracking mode is published on `/tracking/mode` at every frame (0=zero, 1=detection, 2=Kalman, 3=PINN, 4=hold, 5=decay), enabling offline analysis of the fallback behaviour across entire flight recordings.

7.3 PX4 Offboard Interface

The PX4 converter node (`px4_converter_node`) bridges the ROS 2 control output to the PX4 autopilot via the Micro XRCE-DDS agent [11] over Ethernet (Jetson 192.168.0.3 $\leftrightarrow$ Pixhawk 192.168.0.4). The node subscribes to `/yaw_command` (Float64) and publishes three PX4 topics at 10 Hz:

- `/fmu/in/offboard_control_mode`: position-hold mode with yaw override enabled.

- `/fmu/in/trajectory_setpoint`: hover at a fixed NED position ($[0, 0, -5]$ m) with the yaw field set to the current command. The position-hold setpoint maintains altitude and lateral position while the yaw command rotates the vehicle to track the target.
- `/fmu/in/vehicle_command`: arming and mode-switch commands during the startup sequence.

The node implements the PX4 offboard startup sequence: setpoint streaming begins immediately, the offboard mode command is issued after 10 control ticks (1 s), and the arm command follows one tick later. This sequence satisfies the PX4 requirement that offboard setpoints must be received before the mode transition is accepted [10].

A safety watchdog monitors the interval between incoming yaw commands. If no command is received for 5 s (configurable via `safety_timeout`), the node logs a warning. In a flight configuration this timeout would trigger a failsafe transition to position-hold or return-to-launch; in the current HIL validation the PX4 failsafe handler manages this transition autonomously.

7.4 Design Scope and Limitations

The current control implementation is restricted to single-axis (yaw) tracking. Altitude hold and lateral position are delegated to the PX4 position controller via the fixed hover setpoint. This design decision reflects the project's validation strategy: yaw-only control isolates the perception-to-command pipeline from aerodynamic coupling effects, enabling clean evaluation of the detection and prediction subsystems. Extension to multi-axis control (altitude tracking via depth-derived z velocity setpoints and pitch adjustment for viewing angle optimisation) is architecturally supported by the 3D Kalman state but is reserved for free-flight validation in future work.

8 Experimental Setup

8.1 Software-in-the-Loop Environment

Initial validation of the middleware architecture and pursuit control hierarchy was conducted in a Software-in-the-Loop (SITL) environment using Gazebo [13] integrated with the PX4 autopilot flight stack [10], employing a standard X500 quadrotor airframe model. To isolate the ROS 2 middleware and control logic from perception-induced latency, a mock detector node publishes synthetic bounding box and tracking data at a deterministic 30 Hz rate. This isolation enables repeatable validation of message serialization, tracking fallback mode transitions, and command generation without the confounding variables of camera capture and neural network inference.

8.2 Hardware Platform and Safety Protocols

Physical validation was conducted within a designated outdoor flight cage. Initial checkout flights used manual radio-control (RC) to verify motor mapping, center of gravity, and baseline stability of the X500 platform prior to payload integration. Safety protocols included a hardware emergency kill switch on the pilot's RC transmitter, real-time battery voltage telemetry, and physical separation between the flight envelope and operators via the cage netting.

The complete autonomous perception-to-control pipeline was evaluated using a Hardware-in-the-Loop (HITL) architecture: the NVIDIA Jetson Orin [6] runs all perception, tracking, and control nodes while connected to a PX4 FMUv6X flight controller operating in HIL mode (simulated dynamics, real autopilot hardware). The

Intel RealSense D455 [8] provides the camera feed, and all communication between the Jetson and FMU uses Ethernet via Micro XRCE-DDS [11].

8.3 Flight Test Matrix

A structured test matrix evaluates both perception robustness and the end-to-end tracking pipeline:

- **Detector Degradation Matrix:** Evaluates RF-DETR [5] against a YOLOv8n [1] baseline on 1,983 images under four synthetically applied degradations (Gaussian noise, motion blur, low-light, occlusion) at five severity levels (0.0–1.0).
- **HITL System Validation:** Six configurations across two input modalities. Three *synthetic* configurations (Sim A, Sim B, Sim C) exercise the PINN and Kalman fallback hierarchy during programmed detection dropouts at 30 Hz. Three *camera* configurations (640×480 bridge, 1280×720 bridge, 1280×720 direct V4L2) test the DDS middleware and capture pipeline under real sensor loads with TensorRT [12] FP16 inference.

8.4 Data Logging and Analysis

All HITL telemetry, perception outputs, and middleware diagnostics were recorded using the ROS 2 [9] `rosbag2` utility with the MCAP storage plugin. MCAP's chunked, compressed format handles the high throughput of the image transport pipeline (up to 2.7 MB per uncompressed 1280×720 BGR8 frame) and high-frequency control loops without inducing secondary serialization bottlenecks, while maintaining precise timestamp synchronization across the distributed node graph.

8.5 Evaluation Metrics

System performance is quantified using the following metrics:

- **Inference Latency (ms):** End-to-end time from image capture to detection publication, measuring the combined cost of the transport pipeline and TensorRT inference.
- **Detection Rate (%):** Ratio of successfully published detections to expected frames, isolating serialization-induced dropouts from detector failures.
- **Tracking Mode Distribution (%):** Fraction of flight time in each tracking state (Detection, Kalman, PINN, Hold, Decay, Zero), quantifying fallback utilization.
- **mAP@0.5:** Used for the isolated detector degradation matrix to quantify bounding box accuracy under adverse conditions.

8.6 AI Tools Disclosure

During the preparation of this work, the authors used GitHub Copilot for code drafting and large language models (Claude, Gemini) for prose refinement. All AI-assisted content was reviewed, edited, and validated by the authors, who maintain full responsibility for the accuracy and integrity of the final publication.

9 Results

9.1 Degradation Robustness

The key quantitative finding is the detector architecture comparison under synthetic degradation, presented in full in Section 4.5. RF-DETR outperforms YOLOv8n by +39.8 mAP@0.5 points under maximum Gaussian noise and +39.4 points under maximum low-light severity, while both models are comparably degraded by motion blur at high severity. Table 1 and Table 2 summarize the key metrics.

9.2 Hardware-in-the-Loop Validation

The complete perception-to-control pipeline is validated in hardware-in-the-loop (HITL) configuration: the NVIDIA Jetson Orin runs all perception, tracking, and control nodes while connected to a PX4 FMUv6X flight controller operating in HIL mode (simulated dynamics, real autopilot hardware). Six test configurations are evaluated across two input modalities: synthetic (mock detector at 30 Hz with programmed dropouts) and camera (Intel RealSense D455 with RF-DETR TensorRT inference at 1280×720). Table 3 summarizes the results.

Table 3: Hardware-in-the-loop test results across six configurations. Synthetic configurations use a mock detector at 30 FPS; camera configurations use RF-DETR TensorRT on the Jetson Orin with an Intel RealSense D455. Bridge denotes V4L2 bridge capture; Direct denotes native V4L2 capture integrated into the detector node.

Config	Input	Duration (s)	Det. Rate (%)	Latency (ms)	P95 (ms)	Dropouts
<i>Synthetic (mock detector, simulated dropouts)</i>						
Sim A	Mock 30 Hz	82.4	22.9	—	—	4
Sim B	Mock 30 Hz	107.2	21.9	—	—	10
Sim C [†]	Mock + PINN	179.9	21.0	—	—	16
<i>Camera (RF-DETR TensorRT on Jetson Orin)</i>						
Cam 640	Bridge 640×480	182.1	100.0	234.5	598.1	1
Cam 720	Bridge 1280×720	182.3	59.5	233.7	558.2	114
Cam Direct	Direct 1280×720	107.7	100.0	58.8	61.5	0

[†]PINN fallback enabled; 77.5% of tracking time in PINN mode.

9.2.1 Inference Latency and Capture Pipeline

The camera configurations reveal the critical impact of the image transport pipeline on end-to-end latency. Publishing 2.7 MB uncompressed frames (1280×720 BGR8) through ROS 2 DDS middleware limits effective throughput to approximately 1.5 FPS in Python, despite the camera capturing at 30 FPS. The V4L2 bridge configurations (Cam 640 and Cam 720) exhibit 234 ms mean latency with high variance (P95: 558–598 ms) due to DDS serialization contention.

By integrating V4L2 capture directly into the detector node, bypassing DDS image serialization entirely, the Cam Direct configuration achieves 58.8 ms mean latency (median 59.0 ms, P95: 61.5 ms), a $4 \times$ reduction from the bridge architecture. Figure 5 shows the resulting tight unimodal distribution, bounded by the combined cost of V4L2 capture and TensorRT inference. This $10 \times$ reduction in latency variance is critical for real-time pursuit control, where consistent frame-to-frame timing prevents oscillatory yaw commands.

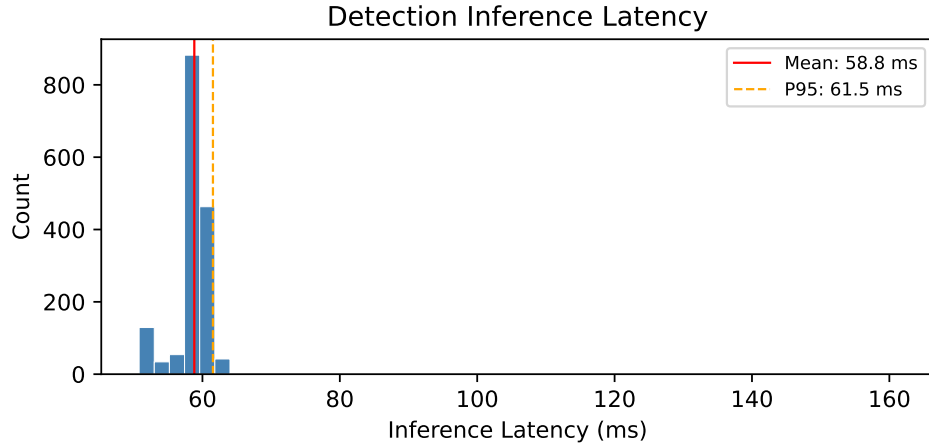


Figure 5: End-to-end detection latency distribution for the direct V4L2 camera configuration on the Jetson Orin. Mean 58.8 ms, P95 61.5 ms; bounded by the combined cost of V4L2 capture and TensorRT inference.

9.2.2 Detection Rate and Dropout Recovery

The detection rate results isolate the DDS serialization bottleneck from detector capability. The Cam 640 (bridge) and Cam Direct configurations both achieve 100% detection rate, demonstrating that RF-DETR reliably detects the target at both resolutions. The Cam 720 bridge configuration drops to 59.5% with 114 dropout events, not because the detector fails, but because DDS cannot transport 2.7 MB frames fast enough. The V4L2 direct capture eliminates this bottleneck entirely: zero dropouts over the 108-second test at full 1280×720 resolution.

The synthetic configurations intentionally introduce dropout periods to exercise the tracking fallback hierarchy. In Sim C, with the PINN enabled, the system spends 77.5% of tracking time in PINN prediction mode, 21.0% in direct detection, and 1.1% in Kalman extrapolation (Figure 6). The PINN maintains continuous yaw tracking commands throughout dropout intervals of up to 16.3 seconds.

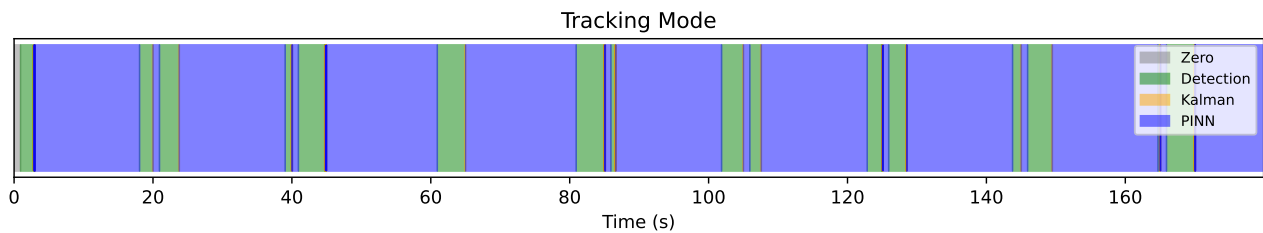


Figure 6: Tracking mode timeline for Sim C (180 s, PINN enabled). Blue regions indicate PINN-predicted tracking during detection dropout; green indicates direct detection. The PINN maintains tracking through extended dropout intervals.

9.2.3 Tracking Mode Comparison

Figure 7 compares tracking mode breakdown across all six configurations. The camera configurations without PINN (Cam 640, Cam 720, Cam Direct) rely on hold and decay modes during dropout, while the PINN-enabled synthetic configuration (Sim C) demonstrates the predictive fallback operating as designed. The early synthetic

configurations (Sim A, Sim B) spend the majority of time in zero mode (68–72%): the Kalman filter bridges short dropouts (3.9–8.6% of tracking time) but resets after 30 consecutive missed updates (≈ 1 s), leaving multi-second gaps untracked without the PINN.

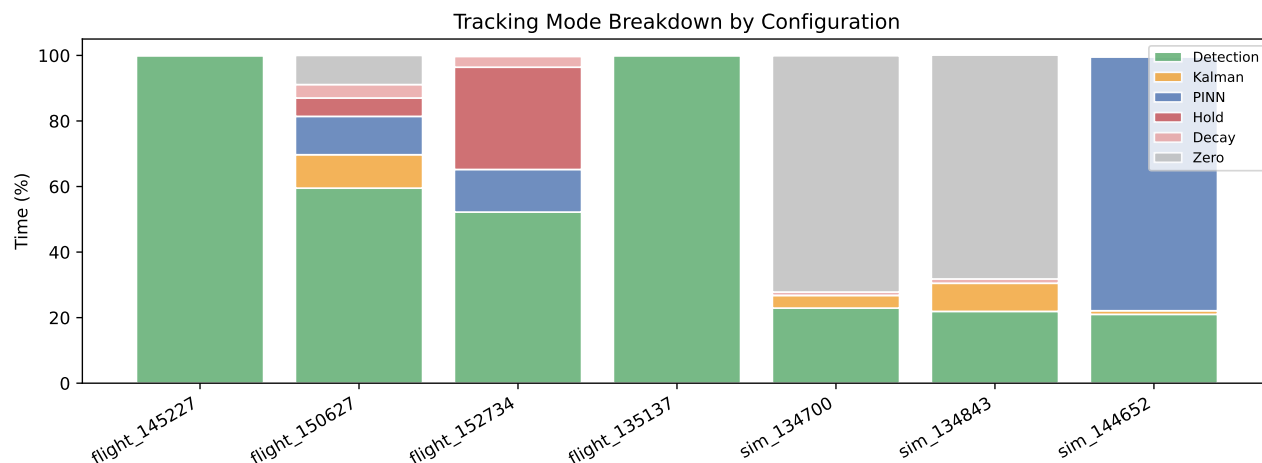


Figure 7: Tracking mode breakdown across all HITL configurations. PINN prediction (blue) dominates in the Sim C configuration where it is enabled; camera configurations fall back to hold/decay modes during dropout.

9.2.4 Yaw Tracking

Figure 8 shows the PX4-reported vehicle yaw and the corresponding yaw rate command for the Cam 720 configuration over a 182-second test. The yaw command signal reflects the fallback hierarchy: smooth tracking during detection, hold during short dropouts, and decay during extended loss. The maximum yaw rate command is clamped at $\pm 57.3^\circ/\text{s}$ as a safety limit.

10 Discussion

10.1 Detector Architecture and Degradation

The degradation evaluation reveals a clear architectural advantage for transformer-based detection under noise and lighting variation. RF-DETR's +39.8 percentage point mAP advantage at maximum Gaussian noise severity is consistent with the self-attention mechanism's ability to aggregate global context: when local pixel regions are corrupted, uncorrupted regions still contribute to detection through the attention weights. CNN detectors like YOLOv8n, which rely on local convolutional kernels, lose access to clean features proportional to the corruption fraction.

The comparable performance of both architectures under motion blur is notable. Blur affects the entire image globally (all pixels shift in the blur direction), eliminating the local-vs-global distinction that gives transformers their advantage under localized corruption. This suggests that blur mitigation requires approaches outside the detector architecture, such as electronic image stabilization or exposure time reduction.

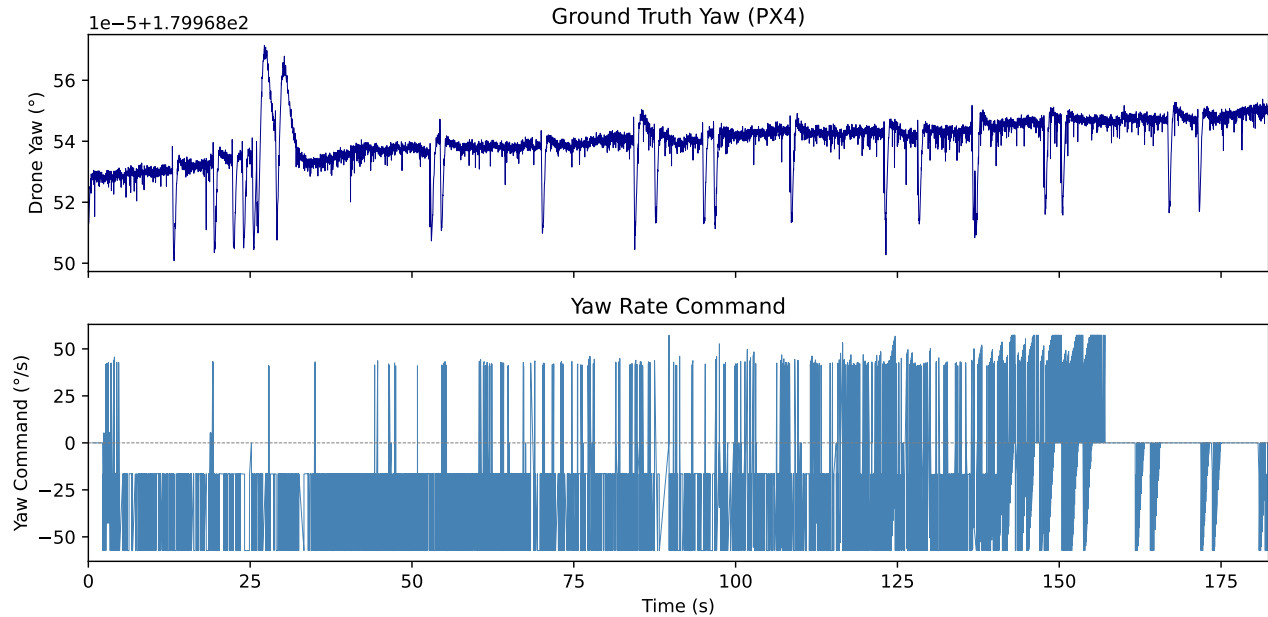


Figure 8: Ground truth yaw (top) and yaw rate command (bottom) for the Cam 720 HITL configuration. The commanded yaw rate reflects the detection/dropout pattern, with commands held or decayed during detection loss.

10.2 HITL Validation and Capture Pipeline Effects

The HITL results highlight the importance of the capture pipeline in determining end-to-end system performance. The V4L2 bridge architecture serializes 2.7 MB BGR8 frames through ROS 2 DDS, introducing 234 ms mean latency with P95 exceeding 558 ms. By integrating V4L2 capture directly into the detector node, bypassing DDS image transport entirely, the Cam Direct configuration reduces mean latency to 58.8 ms (P95: 61.5 ms), a $4\times$ improvement, and eliminates all detection dropouts at 1280×720 . This confirms that the RF-DETR TensorRT engine achieves approximately 15 FPS on the Jetson Orin with latency bounded by V4L2 capture and inference rather than the transport layer, well within real-time control loop requirements. The bridge-induced dropouts in Cam 720 (59.5% detection rate, 114 dropout events) were an artifact of the transport layer, not a limitation of the detector.

10.3 Predictive Tracking During Dropout

The Sim C configuration demonstrates the PINN's role in the fallback hierarchy. During the 180-second test, the system encounters 16 dropout events (longest 16.3 s) and maintains continuous yaw tracking commands throughout via PINN prediction. Without the PINN (Sim A, Sim B), the system spends 68–72% of time in zero mode (no tracking command) during equivalent dropout patterns. This validates the PINN as an effective bridge between detection events, though the current evaluation is limited to synthetic dropout patterns; validation with real camera dropout patterns is ongoing.

10.4 Limitations

Several limitations of the current evaluation should be noted. First, all HITL tests use simulated flight dynamics (PX4 HIL mode); the vehicle does not physically fly during these tests. Aerodynamic effects, vibration-induced

blur, and GPS-denied conditions are not captured. Second, while the direct V4L2 capture configuration achieves 15 FPS at 1280×720 , the camera's native 30 FPS capability is not fully utilized due to a librealSense2 ARM64 compatibility issue requiring the V4L2 fallback path; depth data is unavailable in this mode. Third, the PINN is trained on SITL trajectory data and has not yet been retrained on real flight trajectories. Fourth, the current pipeline tracks a single target class; multi-target association and priority assignment are not addressed.

11 Conclusion

We presented AIRHOUND, a modular perception-to-control pipeline for autonomous UAV target pursuit running on edge hardware. The system makes three contributions relevant to the DS112 track theme of machine learning from challenging data. First, RF-DETR demonstrates a substantial robustness advantage over YOLOv8n under Gaussian noise and low-light degradation (+39.8 and +39.4 percentage points at maximum severity respectively), while both architectures remain vulnerable to severe motion blur. Second, the physics-informed trajectory prediction network provides bounded extrapolation during detection dropout, maintaining continuous tracking commands through dropout intervals of up to 16.3 seconds in HITL testing. Third, the modular ROS 2 architecture enables systematic ablation of each component (detector, Kalman filter, PINN) through runtime configuration.

Hardware-in-the-loop validation on the Jetson Orin with the Intel RealSense D455 confirms real-time performance: the RF-DETR TensorRT engine achieves 58.8 ms mean end-to-end detection latency (P95: 61.5 ms) with 100% detection rate and zero dropouts at 1280×720 when using direct V4L2 capture, and the complete perception-to-control pipeline runs end-to-end with closed-loop yaw tracking through the PX4 flight controller. Future work includes free-flight validation, multi-axis control (altitude and pitch), PINN retraining on real flight trajectories, and multi-target tracking. A further direction is replacing the constant-velocity PINN with a social transformer architecture [16] that jointly reasons about pursuer and target state to produce interaction-aware trajectory predictions; such models have demonstrated strong performance in pedestrian settings and could extend to adversarial drone pursuit given suitable training data from multi-vehicle flight campaigns.

Acknowledgments

This work was conducted at the Engineering Physics Propulsion Lab (EPPL), Embry-Riddle Aeronautical University, under the direction of Dr. Sergey Drakunov. Hardware development was supported by an Embry-Riddle Undergraduate Research (Spark) equipment grant. Travel to the conference was funded by the SPIE Student Conference Support program and the Embry-Riddle Department of Physical Sciences Research Committee.

References

- [1] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8, 2023. URL <https://github.com/ultralytics/ultralytics>.
- [2] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. *arXiv preprint arXiv:2005.12872*, 2020.
- [3] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.

- [4] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- [5] Isaac Robinson, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. RF-DETR: Neural architecture search for real-time detection transformers. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2511.09554>.
- [6] NVIDIA Corporation. Jetson Orin modules, 2024. URL <https://developer.nvidia.com/embedded/jetson-orin>.
- [7] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.
- [8] Intel Corporation. Intel RealSense depth camera D455, 2024. URL <https://www.intel.com/content/www/us/en/products/sku/205847/intel-realsense-depth-camera-d455/specifications.html>.
- [9] Open Robotics. ROS 2 humble hawkbill documentation, 2024. URL <https://docs.ros.org/en/humble/>.
- [10] PX4 Development Team. PX4 autopilot user guide, 2024. URL <https://docs.px4.io/main/en/>.
- [11] eProsima. Micro XRCE-DDS: An XRCE implementation for embedded systems, 2024. URL <https://micro-xrce-dds.docs.eprosima.com/en/latest/>.
- [12] NVIDIA Corporation. NVIDIA TensorRT, 2024. URL <https://developer.nvidia.com/tensorrt>.
- [13] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2149–2154. IEEE, 2004. doi: 10.1109/IROS.2004.1389727.
- [14] Roboflow Community. Drones object detection dataset, 2024. URL <https://universe.roboflow.com/customyolo-k3lvg/drones-romla/dataset/1>. CC BY 4.0 License.
- [15] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.
- [16] Francesco Giuliari, Irtiza Hasan, Marco Cristani, and Fabio Galasso. Transformer networks for trajectory forecasting. In *Proc. International Conference on Pattern Recognition (ICPR)*, pages 10335–10342. IEEE, 2021. doi: 10.1109/ICPR48806.2021.9412190.